# Mini Assessment

API Response Handling

**Name:** M. Hemalatha

**Registration No:** 99230040364

**Department:** CSE (AI & ML)

**College:** Kalasalingam University of Research and Education

**Date:** 11-09-26

## Q1. If an API returns 429, what should your program do?

**Answer:** HTTP status code 429 means that the client has sent too many requests in a given period. The program should not immediately send another request because this may continue the rate-limit problem. Instead, it should wait before retrying, preferably using a backoff strategy. If the server provides a **Retry-After** header, the program should follow the specified waiting time. This makes the application more reliable and respectful of the API's usage limits.

## Q2. If response.json() fails, what should your program do?

**Answer:** The program should not assume that every API response contains valid JSON. The **response.json()** operation should be handled safely so a parsing failure does not crash the application. The program can inspect the response status, content type, or response text to understand the problem. It should then log or report the error and handle the unexpected response gracefully. This is especially important when an API returns an HTML error page or another non-JSON response.

## Q3. If a JSON key is missing, why might .get() be safer than []?

**Answer:** Using **data['key']** with square brackets raises a **KeyError** when the key does not exist. This can interrupt the program if the API response contains optional or missing fields. The **data.get('key')** method is safer because it returns **None** when the key is absent, or a specified default value. This allows the program to handle missing data gracefully and continue execution. Therefore, **.get()** is useful when the presence of a JSON key cannot be guaranteed.